
Search Engine Project

Stage 1: Building the Data Layer

C++ implementation — technical report

Course	Big Data · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria
Academic year	2026/2027
Group name	BigData-Ulpgc
Members	Carlos Falcón Brito — C++ implementation, author of this report Daniel Rodríguez Alonso — Java implementation Daniel Perdomo Medina — Python implementation
Repository	https://github.com/BigData-Ulpgc/stage_1 , folder <code>cpp/</code>
Status	Complete pipeline, offline mode with the sample dataset, structures chosen by configuration, 12 benchmark experiments, 232 tests passing
Date	4 October 2026

Quick check for instructors. From `cpp/`, with a C++20 compiler, CMake, Ninja, SQLite3, libcurl and mongo-cxx-driver installed, and no network: `make`, then `./build/release/search_engine_stage1 pipeline 30 --offline` and `./build/release/search_engine_stage1 search whale island`. Section 6.3 gives the exact commands and the expected output.

Abstract

This report describes the C++ implementation of Stage 1 of the group’s search engine over Project Gutenberg books. Stage 1 builds the *data layer*: a pipeline that downloads books, splits each one into header and body, stores them in a datalake, extracts their metadata into a SQLite datamart and builds an inverted index that answers Boolean AND queries. A control layer records the progress, so an interrupted run resumes without losing or repeating books.

The module offers three datalake layouts (**book**, **range**, **time**) and three persistent inverted-index structures (**monolithic** JSON, **hierarchical** folders, **mongo**), selectable through a configuration file without recompiling. It follows the group’s common contract (**shared/SPEC.md**): it reproduces the reference term counts of the shared dataset exactly and stores byte-identical books. A 15-book sample dataset lets the whole pipeline run offline in under half a second. The 12 benchmark experiments, run on the 200 real books of the dataset, lead to **book + monolithic** for the final system. They also show that the **monolithic** index is the fastest to build and to query, but its update cost and memory grow with the index; that **hierarchical** reserves 73 times its data size in file-system blocks; and that SQLite’s **author** and **title** indexes keep metadata lookups flat as the table grows.

Contents

1	Introduction and objectives	3
1.1	Goal of Stage 1	3
1.2	The group and the common contract	3
1.3	Objectives and scope of this report	3
2	System architecture	4
2.1	The life of a book	4
2.2	Layers and dependencies	5
2.3	Design principles applied throughout	6
2.4	Project organisation	6
2.4.1	Folder tree	6
2.4.2	Build targets and dependencies	7
3	Design decisions	7
3.1	Structures selected for the final implementation	7
3.2	Implementation decisions	8
4	Implementation: modules and classes	9
4.1	Module util : file and text helpers	9
4.2	Module crawler : fetching and splitting	9
4.3	Module datalake : storing the books	9
4.4	Module datamart/metadata : metadata in SQLite	10
4.5	Module datamart/index : the inverted index	10

4.6	Module <code>query</code> : the search	11
4.7	Module <code>control</code> : progress and recovery	11
4.8	Module <code>config</code> : configuration and factories	11
4.9	Module <code>benchmark</code>	12
4.10	The command line	12
5	Testing strategy	12
5.1	MongoDB tests	13
5.2	The sample dataset test	13
5.3	Checks beyond the unit tests	13
6	Setup and execution	13
6.1	Requirements	13
6.2	Build and test	14
6.3	Quick test without network: the sample dataset	14
6.4	Commands	14
6.5	Configuration: changing structures without recompiling	15
6.6	Running the benchmarks	15
7	Benchmarks and results	15
7.1	Methodology	15
7.2	Data	16
7.3	Datalake benchmark	16
7.4	Metadata benchmark	17
7.5	Inverted-index benchmark	18
7.6	Summary: which structure to choose	20
8	Conclusions and future improvements	20
8.1	Conclusions	20
8.2	Limitations and future improvements	20

1. Introduction and objectives

1.1 Goal of Stage 1

The project builds a search engine over books from Project Gutenberg in several stages. Stage 1 builds the data layer that the later stages query. Table 1 lists its parts.

Part	What it does
Crawler	Downloads a book and splits it into header and body at the START/END markers, dropping the license footer.
Datalake	Stores the raw books in one of three folder layouts.
Metadata datamart	Keeps title, author, language and release date of every book in SQLite.
Inverted-index datamart	Maps each term to the sorted ids of the books that contain it, in one of three structures.
Query	Answers a query with the books that contain <i>every</i> term (AND semantics).
Control layer	Records which books are downloaded and which are indexed, so a run can resume.

Table 1: The parts of the Stage 1 data layer.

1.2 The group and the common contract

The same data layer is implemented three times, in Java, Python and C++, one implementation per member. All three follow `shared/SPEC.md`, so they process the same dataset with the same rules and produce equivalent outputs. Table 2 lists what the contract fixes.

SPEC	Topic	What it fixes
§1	Dataset	200 Gutenberg ids in <code>shared/book_ids.txt</code> ; the query workload; the stopwords.
§2	Download and split	The URL, the START/END markers, header and body trimmed, <code>\r\n</code> normalised to <code>\n</code> .
§3	Datalake layouts	<code>book: <ID>/body.txt</code> ; <code>range: 01000-01999/<ID>.body.txt</code> ; <code>time: YYYYMMDD/HH/<ID>.body.txt</code> .
§4	Metadata	Four regular expressions and the SQLite table <code>books</code> , with indexes on <code>author</code> and <code>title</code> .
§5	Tokenizer	ASCII letters and digits only, lower case, tokens of two or more characters, stopwords removed.
§6	Index structures	The monolithic JSON file, the hierarchical folders and the MongoDB collection.
§7	Queries	AND semantics; result in ascending id order.
§8	Control layer	An id is written only after its step succeeded.
§9–10	Benchmarks	The CSV format, 2 warm-ups and 5 measured runs, and the data and sizes of each experiment.

Table 2: What the common contract fixes for every implementation.

1.3 Objectives and scope of this report

- Describe the architecture of the C++ module and how a book moves through it.
- Justify the design decisions, including the structures selected for the final implementation.

- Walk through the modules and their classes.
- Explain how the module is tested, built and run.
- Present and discuss the results of the 12 benchmark experiments.

C++ was chosen for this implementation because the data layer is dominated by text processing, file input/output and in-memory data structures: C++ gives direct control over memory and over every system call, and makes those costs visible in the benchmarks. The full record of how the module was built, step by step and with the reason behind every choice, is in `cpp/docs/DEVLOG.md`; the build and usage details are in `cpp/docs/USER_GUIDE.md`.

2. System architecture

Figure 1 follows one book through the module. Every storage box is an interface with several interchangeable implementations.

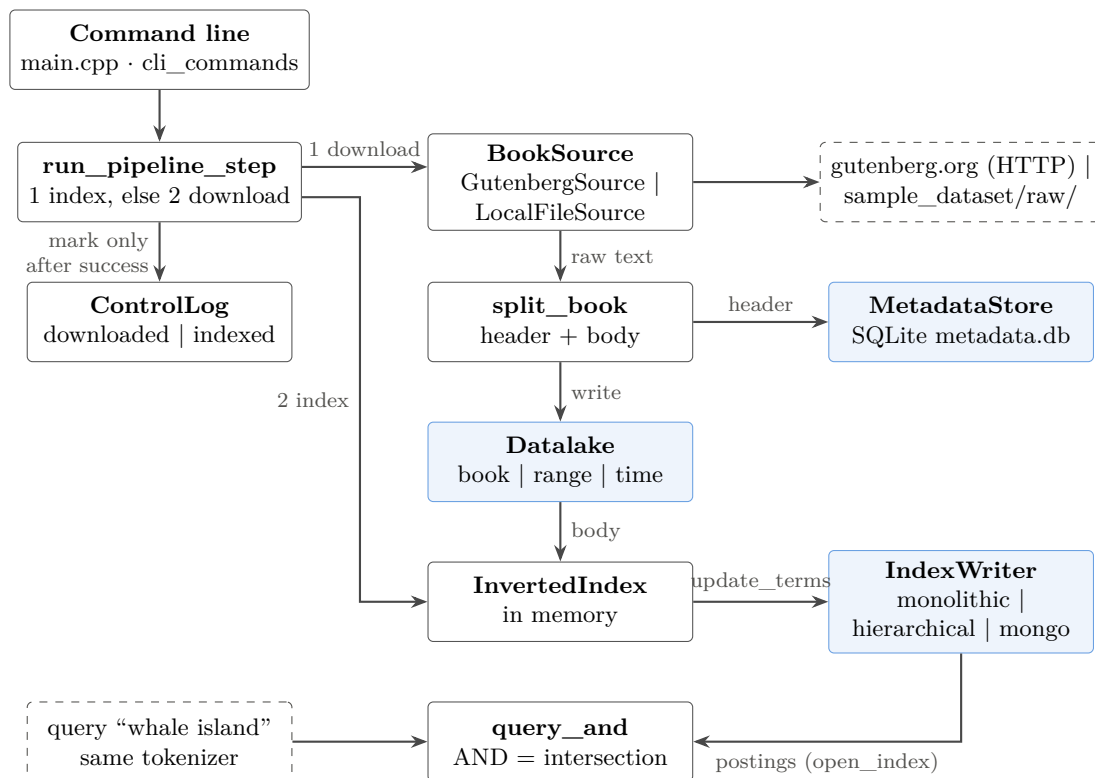


Figure 1: Main components and data flow. `cli_commands` builds every piece from the configuration, using `create_datalake` and `create_index_writer`.

2.1 The life of a book

1. `run_pipeline_step` asks `next_control_action`, which reads only the two control logs. No book is waiting to be indexed, so it picks the first id of `book_ids.txt` that is not downloaded yet, for example 1342.
2. `BookSource::fetch(1342)` returns a `DownloadResult`: `GutenbergSource` downloads the `.txt` over HTTP with `libcurl`, or `LocalFileSource` reads it from the sample dataset. `split_book` finds the markers and returns a `SplitBook` with the header and the body; a book without both markers is discarded.

3. `Datalake::write` stores header and body and returns a `BookLocation`. `extract_metadata` parses the header, and `MetadataStore::insert_book` saves the row together with the two paths.
4. Only if all of this succeeded does `ControlLog::mark` record 1342 in the download log, `downloaded_books.txt`.
5. The next step finds a downloaded book that is not indexed. It reads the body through the path stored in SQLite, tokenizes it, adds the book's distinct terms to the in-memory `InvertedIndex`, and persists only those terms with `IndexWriter::update_terms`.
6. Only then is 1342 appended to `indexed_books.txt`.
7. A query “whale island” is tokenized like a book; `query_and` asks the configured index for the posting lists of `whale` and `island` and intersects them, starting with the shortest.

2.2 Layers and dependencies

Dependencies always point downwards: a module only uses the modules below it (Figure 2). The orchestration code depends on interfaces (`Datalake`, `IndexWriter`, `BookSource`, `HttpClient`, `Clock`); only the command line and the factories know which concrete class is used.

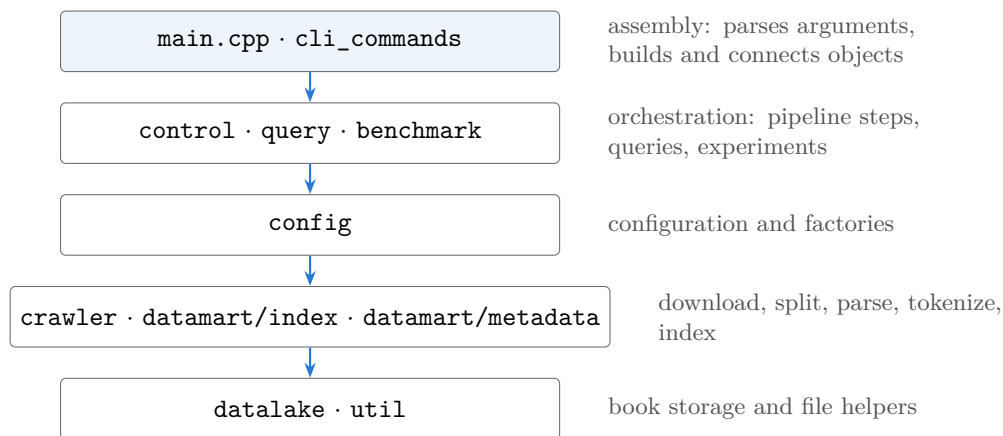


Figure 2: Module layers. Each one only uses the layers below it.

2.3 Design principles applied throughout

Principle	Where	Why
Programming to interfaces	Datalake, IndexWriter, BookSource, HttpClient, Clock	Changing a structure does not touch its users; the tests plug in fakes.
Dependency injection	run_pipeline_step receives every collaborator by reference; TimeBasedDatalake receives its Clock.	Nothing is created behind the scenes; any piece can be replaced in tests and benchmarks.
Factories	create_datalake, create_index_writer, open_index	One configuration name → one class, in a single place.
RAII	MetadataStore (sqlite3*), Statement, CurlHandle, the tests' TempDir	Resources are released on every path out of a function, exceptions included.
Pure decisions	next_control_action, split_book, extract_metadata, tokenize, query_and	The logic that decides is tested without disk or network.
Mark after doing	run_pipeline_step: ControlLog::mark only after the step succeeded.	The control layer never claims what did not happen; at worst a step is repeated.
Value types	SplitBook, BookLocation, StoredBook, StepResult, AppConfig	Plain data without behaviour, copied or moved, never shared by accident.
Fail fast, absence explicit	Configuration checked at start-up; std::optional for a missing marker or book; exceptions for I/O errors.	A wrong configuration stops the program before it writes anything; expected absence is never an error code.

Table 3: Design principles and where they appear in the code.

2.4 Project organisation

2.4.1 Folder tree

The three source trees share one layout, one folder per module, so a header, its source and its tests are always found at the same relative path.

```

cpp/
  CMakeLists.txt, CMakePresets.json, Makefile    build: library, executable, tests
  config.properties                             the structures pipeline and search use
  include/stage1/    util/ crawler/ datalake/ datamart/{metadata,index}/ query/
                                     control/ config/ benchmark/{datalake,index,metadata}/ cli_commands.hpp
  src/              the same folders; main.cpp, cli_commands.cpp
  tests/            the same folders; fakes/ (FakeHttpClient, FakeClock), support/ (TempDir)
  benchmarks/results/  real/{datalake,index}/ and synthetic/metadata/ CSVs (committed)
  docs/             USER_GUIDE.md, DEVLOG.md, this report and its sources (report/)
  data/             written by the pipeline (git-ignored)

```

2.4.2 Build targets and dependencies

Target	Contents
<code>stage1_core</code>	Static library with all the logic: every module except the command line.
<code>search_engine_stage1</code>	The executable: <code>main.cpp</code> and <code>cli_commands.cpp</code> . It alone receives the build-time paths of <code>shared/</code> , <code>data/</code> , <code>benchmarks/</code> and <code>config.properties</code> , so it runs from any folder.
<code>stage1_tests</code>	One GoogleTest executable with the 232 tests, registered in CTest.

Table 4: CMake targets. The presets **release** (benchmarks) and **debug** (development) configure Ninja builds; `make` and `make test` are shortcuts.

Library	Obtained from	Used for
SQLite3	System (macOS SDK, <code>libsqlite3-dev</code>)	The metadata datamart.
libcurl	System	Downloading books over HTTP.
mongo-cxx-driver 4.x	Homebrew or the driver’s sources	The <code>mongo</code> index structure.
nlohmann/json	CMake <code>FetchContent</code>	The monolithic JSON index.
GoogleTest	CMake <code>FetchContent</code>	The test suite.

Table 5: External libraries.

3. Design decisions

3.1 Structures selected for the final implementation

The final system uses **book** for the datalake and **monolithic** for the inverted index, selected in `cpp/config.properties`. The benchmarks of Section 7 support both choices at the project’s 200 books.

Datalake: **book**.

- **Lookup.** The fastest one that survives a restart: $2.52\mu\text{s}$ per book, against $2.92\mu\text{s}$ for **range**. A **book** path is computed from the id alone. **time** cannot compute a book’s folder from its id; this module’s $0.07\mu\text{s}$ comes from an in-memory map that a new process does not have, and a lookup that survives a restart has to scan the day and hour folders.
- **Detection and recovery.** The fastest detection of new books (0.98ms, against 1.16 for **time** and 1.66 for **range**) and the fastest recovery after a crash (19.1ms, against 20.0 and 22.3).
- **Its one loss.** Writing all 200 books takes 228ms, against 192 for **range** and 187 for **time**. The three layouts store identical bytes; **book** reserves 0.6% more disk.
- **When it would change.** **book** adds one folder per book to the datalake’s root (200 today). For a collection the size of the whole Gutenberg catalogue, **range** keeps the direct lookup and caps each folder at 1,000 books.

Inverted index: **monolithic**.

- **Queries.** The fastest: $1.4\mu\text{s}$ per query, against $22.0\mu\text{s}$ for **hierarchical** and $314.6\mu\text{s}$ for

mongo.

- **Build and update.** The fastest to build (1.8s, against 6.5 and 3.4s) and the cheapest to update at this size (300 ms per added book, against 482 and 922 ms).
- **Its costs grow with the index.** Adding a book costs 93, 165 and 300 ms at $N = 50$, 100 and 200 (the whole file is rewritten), and keeping the index open takes 16.3, 28.1 and 54.1 MB. Extrapolating these three sizes in a straight line, **hierarchical** becomes the cheaper index to update at roughly 700 books. Past that point it is the natural next structure: it also beats **mongo** on both queries and updates.

3.2 Implementation decisions

Decision	Alternative discarded	Reason
Each indexed book persists only its own terms (update_terms)	Rewriting the whole index per book	The rewrite made hierarchical over 100 times slower than monolithic ; updating only the changed terms was measured 3–4 times faster.
SQLite rows inserted in batches: one transaction and one prepared statement per batch, all or nothing	One commit per row	Committing every row was slow and noisy; batching was about 2.5 times faster, and a failed batch leaves nothing behind.
MongoDB updates sent as one unordered bulk write	One update_one per term (about 7,900 round trips per book)	An index update went from 3,421 ms to about 300 ms per book.
Index rebuilt in memory at start-up, then compared with the stored structure	Trusting that the stored structure exists	A missing, partial or stale structure is rewritten whole. Found by testing: 5 stale documents in a Mongo collection made search answer from them.
The time layout's locate remembers the paths it wrote	Scanning the day and hour folders	Constant-time lookups. Indexing reads the body path from SQLite, so the pipeline never needs locate across runs.
Structures chosen in config.properties , overridable with -Dkey=value	Hardcoded structures; a flag on every command	One file keeps pipeline and search on the same index; a typo stops the program at start-up.
Paths fixed at build time	Paths relative to the working directory	The executable finds its files from any folder.
Offline source over sample_dataset/raw/	Network-only pipeline	The whole pipeline runs in under half a second without network, and its output is compared byte for byte with the expected split.
Reproducible benchmarks: a simulated clock, a fixed-seed random generator, untimed preparation, verification before reporting	Real clock and ad-hoc randomness	Every repetition builds the same data, and every implementation makes the same “random” choices.

Table 6: Implementation decisions. `cpp/docs/DEVLOG.md` records each one with its measurements.

4. Implementation: modules and classes

This section walks through the modules from the bottom layer up. All code lives in the namespace `stage1`.

4.1 Module `util`: file and text helpers

`write_text_file` / `read_text_file` free functions

Write a string to a file, creating any missing parent folder first, and read a whole file back. Both throw `std::runtime_error` with the path when the file system fails.

`collect_body_header_pairs`, `count_files_with_suffix`, `trim` free functions

Shared by the datalake layouts and the benchmarks: list the ids that have both a body and a header in a folder, count files by suffix, and trim whitespace.

Design rationale. Every layout writes files the same way; keeping that in one place means a single, tested definition of “write a file”.

4.2 Module `crawler`: fetching and splitting

`HttpClient` interface

One method, `get(url)`, returning a `DownloadResult`. `CurlHttpClient` implements it with `libcurl` (following redirects, with an RAII `CurlHandle`); the tests use `FakeHttpClient`.

`DownloadResult` value type

Either `success(text)` or `failure(message)`: a failed download is an expected outcome, not an exception.

`BookSource` interface

`fetch(book_id)`. `GutenbergSource` builds the SPEC URL and calls an `HttpClient`; `LocalFileSource` reads `pg<ID>.txt` from a folder, which is the offline mode.

`split_book` free function

Normalises `\r\n` to `\n`, finds the START and END markers (“THE” or “THIS” variant), and returns a `SplitBook` with the trimmed header and body, or `std::nullopt` when a marker is missing.

Design rationale. Where a book comes from is the only difference between the online and offline modes: both are `BookSources`, and nothing after `fetch` knows which one it was given.

4.3 Module `datalake`: storing the books

`Datalake` interface

Three operations: `write(id, header, body)` returns a `BookLocation` with both paths; `locate(id)` finds a book; `list_book_ids()` walks the folders and returns every id with both files.

`BookBasedDatalake` layout

`<root>/<ID>/body.txt` and `header.txt`. The path is a pure function of the id.

RangeBasedDatalake layout

`<root>/<INI>-<FIN>/<ID>.body.txt`, with `range_folder_name` computing the five-digit range from the id.

TimeBasedDatalake layout

`<root>/YYYYMMDD/HH/<ID>.body.txt`, the folder taken from an injected `Clock`: `SystemClock` in production, `FakeClock` in the tests and a simulated clock in the benchmarks. Since the path depends on *when* the book was written, `locate` uses a map of the paths this instance wrote.

Design rationale. The interface is what the benchmarks plug the three layouts into; `list_book_ids` reads only the disk, so it works from a new process and is what recovery and change detection rely on.

4.4 Module datamart/metadata: metadata in SQLite**extract_metadata** free function

Applies the four SPEC regular expressions (`std::regex`, multiline) to a header: first match, first line of the value, trimmed. A missing field becomes `std::nullopt` and is stored as SQL NULL.

MetadataStore class, owns a `sqlite3*`

Creates the `books` table and its two indexes on first open. `insert_book` inserts or replaces one row; `insert_books` inserts a batch in one transaction with one prepared statement, rolling the whole batch back if a row fails; `count`, `find_by_id`, `find_by_author` and `find_by_title` (exact match, ascending ids) read it back as `StoredBook` values.

Design rationale. An RAII `Statement` wrapper finalises every prepared statement on every path; the transaction methods are private, so a caller cannot leave a transaction open.

4.5 Module datamart/index: the inverted index**tokenize, load_stopwords** free functions

The SPEC tokenizer, byte by byte: A-Z lower-cased, a-z and 0-9 kept, any other byte a separator, tokens shorter than two dropped, stopwords removed.

InvertedIndex class

An `std::unordered_map` from term to `std::set<int>`: `add_book`, `postings` (sorted, no duplicates), `term_count` and `entries` (every term with its postings, in term order).

IndexWriter interface

`write(index)` makes a structure match the whole index; `update_terms(index, terms)` persists only the given terms. The default `update_terms` calls `write`, which is all a single file can do.

MonolithicIndexWriter structure

One JSON object `{"term": [ids]}` with `nlohmann/json`: keys sorted, compact output.

HierarchicalIndexWriter structure

`inverted_index/<FIRST LETTER>/<term>.txt`, one id per line. `write` replaces the folder; `update_terms` rewrites only the files of the given terms.

MongoIndexWriter structure

One document `{term, postings}` per term with a unique index on `term`. `write` replaces the collection with `insert_many`; `update_terms` sends one unordered bulk write of upserts. The real index, the benchmarks and the tests use separate databases.

Readers free functions

`monolithic_postings_fetcher` parses the JSON once; `hierarchical_postings_fetcher` reads one file per lookup; `mongo_postings_fetcher` runs one query per term. Each returns the function `query_and` takes.

Design rationale. Readers return a plain `std::function`, so querying an on-disk structure and querying the in-memory index are the same code.

4.6 Module query: the search**query_and** free function

Takes the query's terms and a postings source, sorts the posting lists by length, and intersects them starting from the shortest, stopping as soon as the result is empty. The result is sorted ascending.

4.7 Module control: progress and recovery**ControlLog** class

An append-only file of ids: `contains`, `mark` and `ids` (sorted).

next_control_action free function

Pure: from the candidate ids and the two logs, decides `IndexBook` (the first downloaded book not indexed), else `DownloadBook` (the first not downloaded), else `Nothing`.

run_pipeline_step free function

Performs that decision against the components it receives and returns a `StepResult`: the decision, whether it completed, and why not.

Design rationale. Indexing goes first, so downloaded books never pile up unindexed; and marking only after success is what makes an interrupted run safe to resume.

4.8 Module config: configuration and factories**AppConfig**, **load_config** value type and function

`datalake.structure` and `index.structure`: defaults, then `config.properties` if it exists, then `-Dkey=value` overrides. An unknown key or value throws `std::invalid_argument`.

create_datalake factory

The only place that turns a layout name into a class. A `time` datalake owns the clock it is given through a private base class constructed before it, so the factory returns a plain `std::unique_ptr<Datalake>`.

create_index_writer, open_index, index_exists, stored_term_count factories

The write and read sides of each index structure at its SPEC location, and the checks the pipeline uses at start-up.

4.9 Module benchmark

measure_elapsed_ms, BenchmarkResult shared

Runs an untimed setup and a timed operation, 2 warm-ups and 5 measured runs with a steady clock; the rows are written in the SPEC CSV format.

JavaRandom, SimulatedClock reproducibility

JavaRandom is a fully specified 48-bit linear congruential generator, the published algorithm of `java.util.Random`, seeded with 42 and checked against reference values. The output of the standard `<random>` distributions depends on the standard library, so they would give a different lookup order and query workload with `libc++` and with `libstdc++`; this generator gives the same sequence on every platform. **SimulatedClock** advances six minutes per call, so the `time` layout gets ten books per hour folder in every repetition.

datalake/, index/, metadata/ experiments

One file per experiment plus a support module each: `fresh_datalake`, `tokenize_all`, `build_index` and `verify_index`, `synthetic_metadata` and `fresh_metadata_store`.

Design rationale. Every experiment verifies before it reports: a written index must answer like the in-memory one, every lookup must find its book, a recovery must leave nothing lost or duplicated.

4.10 The command line

main.cpp entry point

Parses `-Dkey=value` overrides and the command, loads the configuration and dispatches.

cli_commands commands

`pipeline`, `search`, `status`, `config` and `benchmark`: each builds the real components (paths fixed at build time) and drives the modules above.

5. Testing strategy

The suite has 232 GoogleTest tests (Table 7), run with `make test`. No test uses the network or the real `data/` folder: they use `FakeHttpClient`, `FakeClock` and temporary folders that are deleted when the test ends.

Area	Tests	What they check
benchmark/	71	Rows, units and order of every experiment; <code>JavaRandom</code> against reference values; the simulated clock; each experiment's verification
datamart/index/	49	Tokenizer and stopwords; the in-memory index; every writer and reader, Mongo included
datalake/	25	Paths, <code>locate</code> and <code>list_book_ids</code> of the three layouts
control/	24	The control decision, the logs, and complete pipeline steps, failures included
datamart/metadata/	22	The four regular expressions; inserts, batches (all or nothing) and lookups
crawler/	19	Marker split, the download client, the offline source
config/	12	Configuration precedence and errors; both factories
query/	7	AND intersection and its edge cases
Smoke	3	The build links and runs

Table 7: The test suite by area.

5.1 MongoDB tests

The 11 tests that need MongoDB use their own database, `search_engine_test`, and are skipped when no server answers, so the suite passes on a machine without Docker. Against the group's container (MongoDB 8.2.12) all of them pass.

5.2 The sample dataset test

`LocalFileSource` is tested on the real `sample_dataset/raw/` files: splitting each of the 15 books must reproduce `sample_dataset/book/<ID>/` byte for byte. A full offline run checks the same at the level of the whole datalake (Section 6.3).

5.3 Checks beyond the unit tests

- The index built from the N books with the lowest ids has exactly the SPEC reference counts: 58,834 / 78,820 / 129,356 terms and 360,970 / 759,087 / 1,581,064 postings for $N = 50$ / 100 / 200, the same counts the other two implementations produce.
- The datalake stores the same 128,980,349 bytes as the other implementations.
- A test that failed only on Linux, because it compared two allocations of a few hundred bytes, was reproduced in an Ubuntu 24.04 (glibc 2.39) container and fixed.

6. Setup and execution

6.1 Requirements

- macOS: `xcode-select --install`, then `brew install cmake ninja mongo-cxx-driver`.
- Linux (Ubuntu or Debian): the distribution's packages `build-essential`, `cmake`, `ninja-build`, `libsqlite3-dev` and `libcurl4-openssl-dev`, plus `mongo-cxx-driver 4.x`.
- CMake downloads `nlohmann/json` and `GoogleTest` on the first build.
- MongoDB is optional; without a server its tests and benchmark rows are skipped.

6.2 Build and test

From the `cpp/` folder, make configures and builds the `release` preset with Ninja, and `make test` runs the 232 tests:

```
make
make test
B=./build/release/search_engine_stage1
```

6.3 Quick test without network: the sample dataset

Starting from an empty `data/` folder, the offline pipeline processes the 15 sample books in 30 steps (one download and one indexing per book):

```
$B pipeline 30 --offline
[pipeline] offline: reading books from ".../sample_dataset/raw"
[pipeline] downloaded book 1342
[pipeline] indexed book 1342
[pipeline] downloaded book 84
[pipeline] indexed book 84
...
[pipeline] downloaded book 76
[pipeline] indexed book 76
$B search whale island
3 book(s) matching all of: whale island
 76  Adventures of Huckleberry Finn
 84  Frankenstein; or, the modern prometheus
2701 Moby Dick; Or, The Whale
$B status
dataset:    200 book id(s) in shared/book_ids.txt
downloaded: 15
indexed:    15
pending:    0
```

The 30 steps take 0.36s on the development machine. `pending` counts the books downloaded but not indexed yet. After this run, `diff -r data/datalake/book ../sample_dataset/book` prints nothing: the datalake is byte-identical to the expected split.

6.4 Commands

Command	What it does
<code>pipeline <N></code>	Up to N steps over the 200 books; each downloads or indexes one book (<code>pipeline 400</code> processes the whole dataset).
<code>pipeline <N> --offline</code>	The same, reading the 15 books of <code>sample_dataset/raw/</code> .
<code>search <words...></code>	AND search on the configured index, with each book's title.
<code>status</code>	Books in the dataset, downloaded, indexed and pending.
<code>config</code>	The configuration file used and the active structures.
<code>benchmark <experiment></code>	One of the 12 experiments; writes its CSV under <code>benchmarks/results/</code> .

Table 8: Commands of `search_engine_stage1`. Exit code 0 on success, 1 on a usage or configuration error or a failed step.

6.5 Configuration: changing structures without recompiling

The file `cpp/config.properties` selects the `book` datalake and the `monolithic` index (keys `datalake.structure` and `index.structure`). Any key can be changed for one run, before the command:

```
$B -Dindex.structure=hierarchical pipeline 400
$B -Dindex.structure=hierarchical search whale island
$B config
```

Each layout has its own folder, `data/datalake/<layout>/`. When the chosen index does not hold every indexed book, the next `pipeline` run writes it whole first. The `mongo` structure needs the group's MongoDB, started with `docker compose up -d` from the repository root.

6.6 Running the benchmarks

After downloading the dataset once (`$B pipeline 400`), each experiment is one call:

```
$B benchmark datalake_storage
$B benchmark index_build
$B benchmark metadata_query
```

The datalake and index experiments read the downloaded books; the index experiments run at $N = 50, 100$ and 200 by themselves; the metadata experiments generate their own rows. Each CSV goes to `benchmarks/results/real/<category>/` or `synthetic/metadata/`.

7. Benchmarks and results

7.1 Methodology

- Every timed experiment runs 2 discarded warm-ups and 5 measured runs; the tables give the median of the five.
- Preparation is never timed. Before each run the setup empties the datalake, the index or the database, and the books are tokenized before any index experiment starts.
- Choices that could vary between runs are fixed. The lookup order and the metadata query workload come from a fixed-seed generator, and the `time` layout uses the simulated clock.
- Every experiment verifies its result before reporting it. Any mismatch stops the benchmark.
- The datalake and metadata experiments ran on macOS (Apple Silicon); the index experiments ran on Linux with native Docker for MongoDB.

7.2 Data

Experiments	Structures	Data
<code>datalake_write</code> , <code>_lookup</code> , <code>_incremental</code> , <code>_recovery</code> , <code>_storage</code>	<code>book</code> , <code>range</code> , <code>time</code>	The 200 real books
<code>index_build</code> , <code>_query</code> , <code>_update</code> , <code>_memory</code> , <code>_disk</code>	<code>monolithic</code> , <code>hierarchical</code> , <code>mongo</code>	The 50, 100 and 200 real books with the lowest ids
<code>metadata_insert</code> , <code>_query</code>	<code>sqlite</code> , <code>sqlite_no_index</code>	1,000, 10,000 and 100,000 generated rows: book i has title “Title $i/2$ ” and author “Author $i/10$ ”

Table 9: Experiments and data. The generated metadata gives every author 10 books and every title 2, whatever the size, so a larger N grows the table, not each answer.

7.3 Datalake benchmark

Layout	Write (ms)	Throughput (books/s)	Lookup (μ s/book)	Detect new (ms)	Recover (ms)
book	228.4	876	2.52	0.98	19.1
range	191.6	1,044	2.92	1.66	22.3
time	186.8	1,071	0.07	1.16	20.0

Table 10: Datalake timings, 200 books. “Detect new” finds the last 10% of the books among all the ids; “Recover” repairs every tenth book (20) after a simulated crash left its body as an unfinished `.tmp` file: every layout recovered the 20 books, with 0 lost and 0 duplicated.

Layout	Files	Folders	Fullest folder	Bytes	Allocated bytes
book	400	200	200	128,980,349	130,883,584
range	400	47	220	128,980,349	130,256,896
time	400	21	20	128,980,349	130,150,400

Table 11: Datalake storage, 200 books. Allocated bytes round every file up to whole 4 KB blocks and count one block per folder.

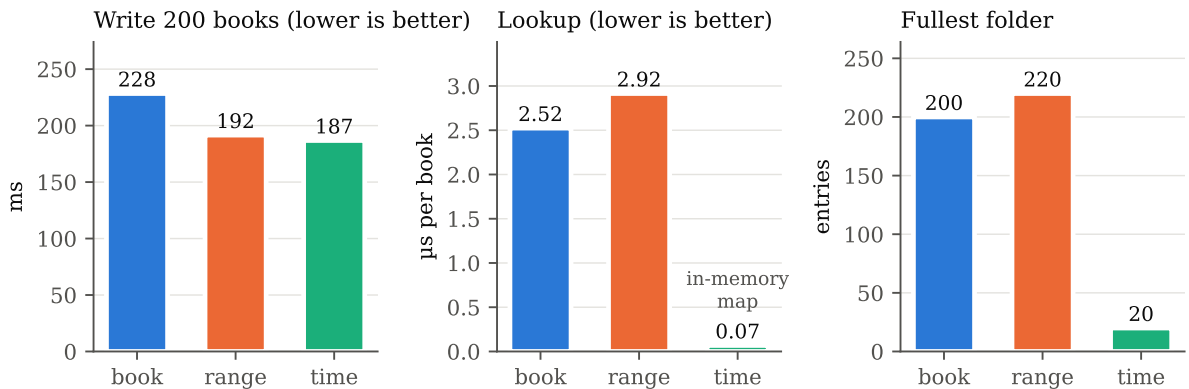


Figure 3: Datalake: writing, lookup and the fullest folder, 200 books.

- **Writing** is dominated by the file system: the three layouts write the same 129 MB in 400 files, and their times differ by under 20%.
- **Lookup** separates the layouts by design. **book** and **range** compute the path from the id and only check that both files exist. **time** cannot; its $0.07\ \mu\text{s}$ is an in-memory map of the paths this instance wrote, so it measures a hash lookup, not the layout.
- **Folder size** is where **time** wins: at most 20 entries per folder, because the folders follow the arrival rate rather than the ids. **book** has one root entry per book, and **range** already holds 220 files in its fullest folder, up to 2,000 once a range fills.
- **Recovery** is safe in all three: the 20 damaged books were found through `list_book_ids` and written again, with nothing lost or duplicated.

7.4 Metadata benchmark

Variant	Rows	Insert (rows/s)	By id (μs)	By author (μs)	By title (μs)
sqlite	1,000	462,330	8.1	12.3	9.5
	10,000	507,759	8.8	13.1	10.5
	100,000	454,570	9.0	15.4	12.7
sqlite_no_index	1,000	932,219	7.9	44.3	41.3
	10,000	1,005,172	8.8	349.8	327.5
	100,000	915,179	9.3	5,869.3	5,726.1

Table 12: Metadata: insertion in batches of 1,000 rows, and the mean time per query over 1,000 queries of each type.

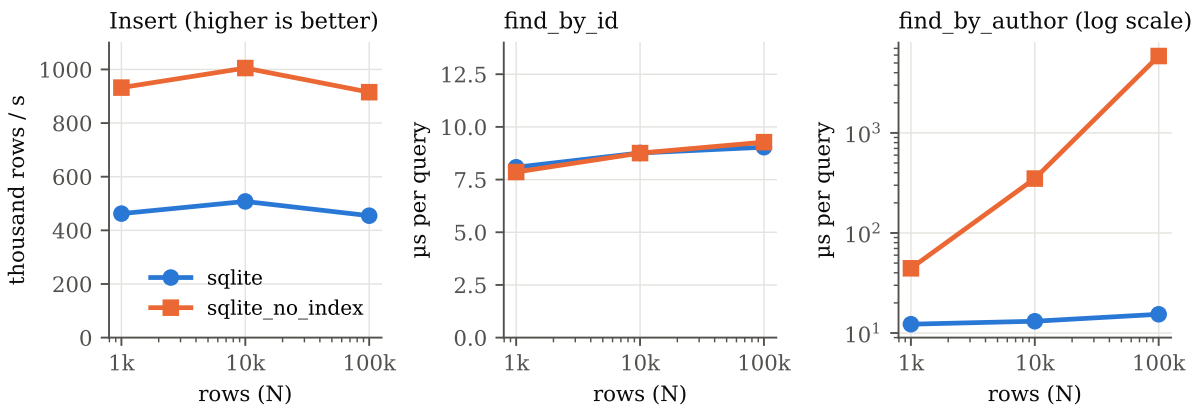


Figure 4: Metadata: insertion and lookups as the table grows (logarithmic row axis).

- **The indexes are what keep lookups flat.** With them, a lookup by author takes 12–15 μs whatever the size; without them it grows with the table, to 5.9 ms at 100,000 rows, about 380 times slower.
- **Lookups by id do not depend on them:** the primary key serves them at 8–9 μs in both variants.
- **Their price is paid on insertion:** maintaining the two indexes halves the insertion rate, from about 930,000 to about 460,000 rows per second.

7.5 Inverted-index benchmark

Structure	Build (ms)			Query (μ s)		
	$N = 50$	100	200	50	100	200
monolithic	282	709	1,849	0.85	0.97	1.42
hierarchical	2,194	3,482	6,546	14.5	16.1	22.0
mongo	968	1,683	3,359	266.6	235.3	314.6

Table 13: Building the whole index, and the mean time of an AND query over 100 rounds of the shared workload.

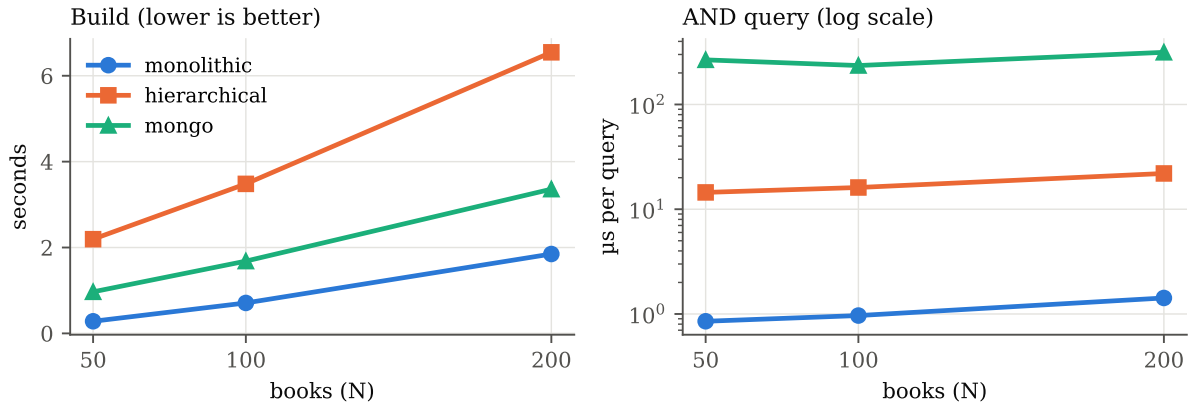


Figure 5: Index build and query time as the collection grows.

Structure	Add one book (ms)			Memory (MB)		
	$N = 50$	100	200	50	100	200
monolithic	93	165	300	16.3	28.1	54.1
hierarchical	330	386	482	0.0	0.0	0.0
mongo	645	685	922	0.01	0.01	0.01

Table 14: Adding the last 10% of the books one at a time to an index of the rest, and the memory in use after opening a written structure for reading (`heap_after_open`).

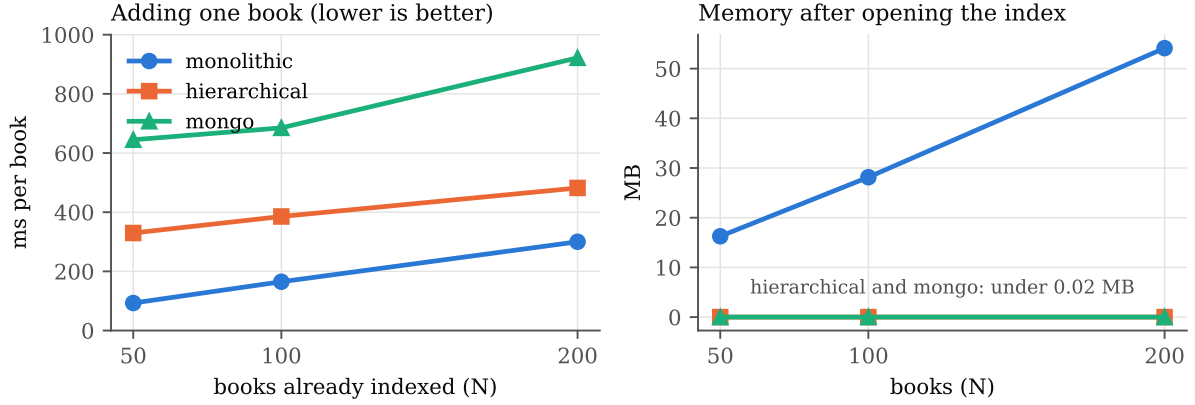
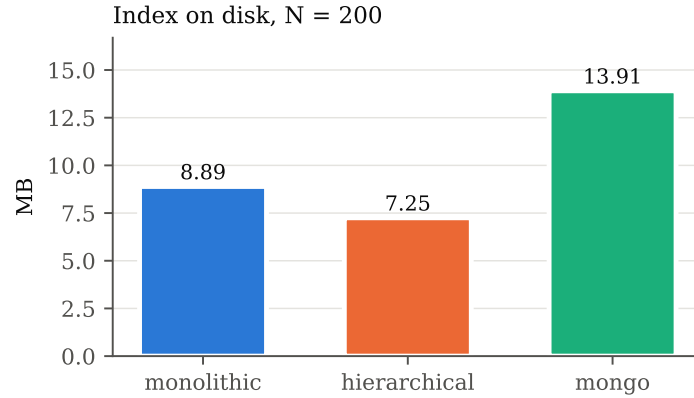


Figure 6: Cost of adding a book, and memory held by an opened index.

Structure	Bytes	Files	Allocated bytes	Allocated / bytes
monolithic	8,893,537	1	8,900,608	1.0
hierarchical	7,251,967	129,356	529,993,728	73.1
mongo	13,910,016	—	—	—

Table 15: The index on disk, $N = 200$: 129,356 terms and 1,581,064 postings in every structure. For **mongo**, the bytes are the collection’s storage plus its indexes, as reported by MongoDB.Figure 7: Index size on disk, $N = 200$.

- **monolithic is the fastest index to build and to query:** $1.4 \mu\text{s}$ per query at $N = 200$, 15 times faster than **hierarchical** and over 200 times faster than **mongo**, whose every lookup is a round trip to the server.
- **But it is the only structure whose costs grow with the whole index.** Adding a book rewrites the entire JSON file ($93 \rightarrow 300$ ms as N quadruples), and opening it loads every posting list ($16 \rightarrow 54$ MB). **hierarchical** and **mongo** touch only the new book’s terms and keep almost nothing in memory.
- **hierarchical has the smallest data but the largest footprint.** Its 7.25 MB of postings sit in 129,356 small files, and every file takes at least one 4 KB block: 530 MB reserved, 73 times the data.

- **Building always holds the whole index in memory:** about 92–97 MB at $N = 200$ for any structure, because every writer persists one complete in-memory `InvertedIndex`.

7.6 Summary: which structure to choose

Part	Choice	Because
Datalake	<code>book</code>	Fastest lookup that survives a restart, fastest change detection and recovery; identical bytes. Move to <code>range</code> for a far larger collection, to bound the folders.
Inverted index	<code>monolithic</code>	Fastest build, queries and updates at 200 books. Move to <code>hierarchical</code> at roughly 700 books, where its per-term updates become cheaper and the monolithic file’s memory keeps growing.
Metadata	<code>sqlite</code> with both indexes	Lookups by author and title stay flat as the table grows, for half the insertion rate.

Table 16: Structures selected for the final implementation.

8. Conclusions and future improvements

8.1 Conclusions

- The C++ module implements the whole Stage 1 data layer: downloading and splitting, three datalake layouts, SQLite metadata, three index structures, AND queries and a control layer that resumes without losing or repeating books.
- It follows the common contract exactly: byte-identical split books, the reference term and posting counts, and the SPEC result format for all 12 experiments.
- Interfaces, factories and dependency injection let every structure be swapped by configuration, and let 232 tests run without network or real data.
- The benchmarks justify `book` + `monolithic` at this scale and say when to change: `monolithic`’s update cost and memory grow with the index, and `hierarchical`’s per-term files waste disk blocks but keep updates and memory bounded.
- Running the experiments on real data found and fixed real problems: missing transaction batching in SQLite, a whole-index rewrite on every update, a hierarchical index that left stale files, per-term round trips to MongoDB, and a stale collection a plain existence check trusted.

8.2 Limitations and future improvements

- **time lookups last one process.** A lookup that survives a restart would scan the day and hour folders, or keep a persistent id-to-path map.
- **Datalake writes are not atomic.** The module relies on the control layer’s write-first, mark-after rule; writing to a temporary file and renaming it would also protect a single file.
- **Start-up cost grows with the dataset.** The in-memory index is rebuilt from every indexed book on each run; reading it back from the persisted structure would avoid that.
- **The command-line wiring has no unit tests,** and five pipeline tests share temporary folder names, so the suite runs sequentially.

- **Each MongoDB call opens a new connection;** keeping one client per writer would remove that cost.
- **Only the two structure keys are configurable;** paths, the MongoDB URI and HTTP timeouts are not, and downloads set no timeout.
- **Growth.** Past a few hundred books the index should move to **hierarchical**, and a collection the size of the Gutenberg catalogue would call for the **range** layout.